



Uninformed & Informed Search

Search Strategies · BFS · DFS · IDS · UCS · Greedy Best-First · A*

Course No: 0613-401

Course Title: Artificial Intelligence

Ekramul Haque Amin

Lecturer

Department of Artificial Intelligence and Data Science

Dhaka International University (DIU)

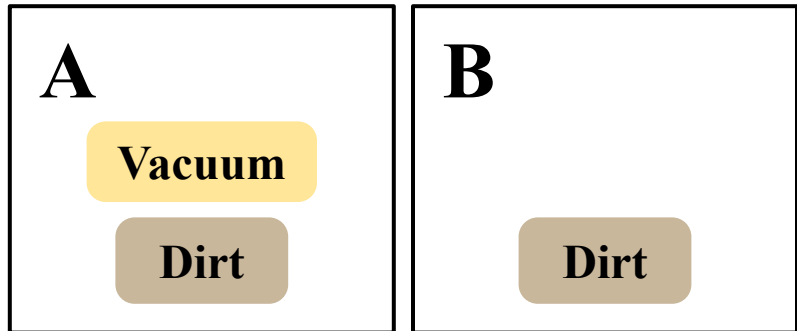
Today's Content

- ▶ **Recap: problem-solving agents and problem formulation**
- ▶ **Searching for solutions: search tree, frontier, performance**
- ▶ **Uninformed search: BFS, DFS, DLS, IDS, UCS, Bidirectional**
- ▶ **Comparing uninformed strategies + class exercise**
- ▶ **Informed search: heuristic functions**
- ▶ **Greedy Best-First Search and A* Search**
- ▶ **Admissible heuristics and 8-puzzle heuristics**

Recap: Problem-Solving Agents

- ▶ A problem-solving agent works in four steps: goal formulation → problem formulation → search → execution.
- ▶ A problem has five components: initial state, actions, transition model, goal test, path cost.
- ▶ Example 1: route from Dhaka to Chattogram — two routes, 340 km (via Noakhali) and 275 km (via Feni & Sitakunda).
- ▶ Example 2: the 8-puzzle — only $9!/2 = 181,440$ states are reachable because a move never changes the parity of the inversion count.
- ▶ Today: how to search the state space — first blind (uninformed) search, then search guided by a heuristic (informed search).

Example 3: Vacuum World as a Search Problem



- ▶ **States**: agent location $\times$ dirt in each square = $2 \times 2 \times 2$ = **8 states**
- ▶ **Initial state**: any state
- ▶ **Actions**: Left, Right, Suck
- ▶ **Transition model**: Suck removes dirt; Left/Right move the agent
- ▶ **Goal test**: both squares clean
- ▶ **Path cost**: 1 per action
- ▶ **Solution** from the state shown: Suck $\rightarrow$ Right $\rightarrow$ Suck

Searching for Solutions

- ▶ **Search tree**: root = initial state; branches = actions; nodes = states in the state space.
- ▶ **Expanding** a node: apply every legal action to generate its child nodes.
- ▶ **Frontier** (open list / agenda): all leaf nodes available for expansion.
- ▶ **Explored set** (closed list): states already expanded — used to avoid repeated states and loops.
- ▶ A **node** stores: state, parent node, action, path cost $g(n)$, depth.
- ▶ **Search strategy** = the rule for choosing which frontier node to expand next.

General Search Algorithm

```
function GRAPH-SEARCH(problem) returns a solution or failure
  frontier ← { initial state }; explored ← { }
  loop do
    if frontier is empty then return failure
    node ← REMOVE(frontier)           // choice depends on strategy
    if GOAL-TEST(node) then return solution(node)
    add node.state to explored
    for each action: child ← CHILD-NODE(node, action)
      if child not in frontier or explored: add child to frontier
```

- ▶ Which node is removed from the frontier defines the type of search and can hugely change its efficiency.
- ▶ Tree search omits the explored set, so it may revisit states and loop forever.

Measuring Search Performance

Criterion	Question it answers
Completeness	Is the algorithm guaranteed to find a solution when there is one?
Optimality	Does it find the solution with the lowest path cost?
Time complexity	How long does it take? (number of nodes generated)
Space complexity	How much memory does it need? (maximum nodes stored)

► Complexity is written using:

- b = branching factor (maximum number of successors of a node)
- d = depth of the shallowest goal node
- m = maximum length of any path in the state space

Uninformed vs Informed Search

Uninformed (Blind) Search

- ▶ No information beyond the problem definition
- ▶ Can only generate successors and test for the goal
- ▶ Examples: BFS, DFS, UCS

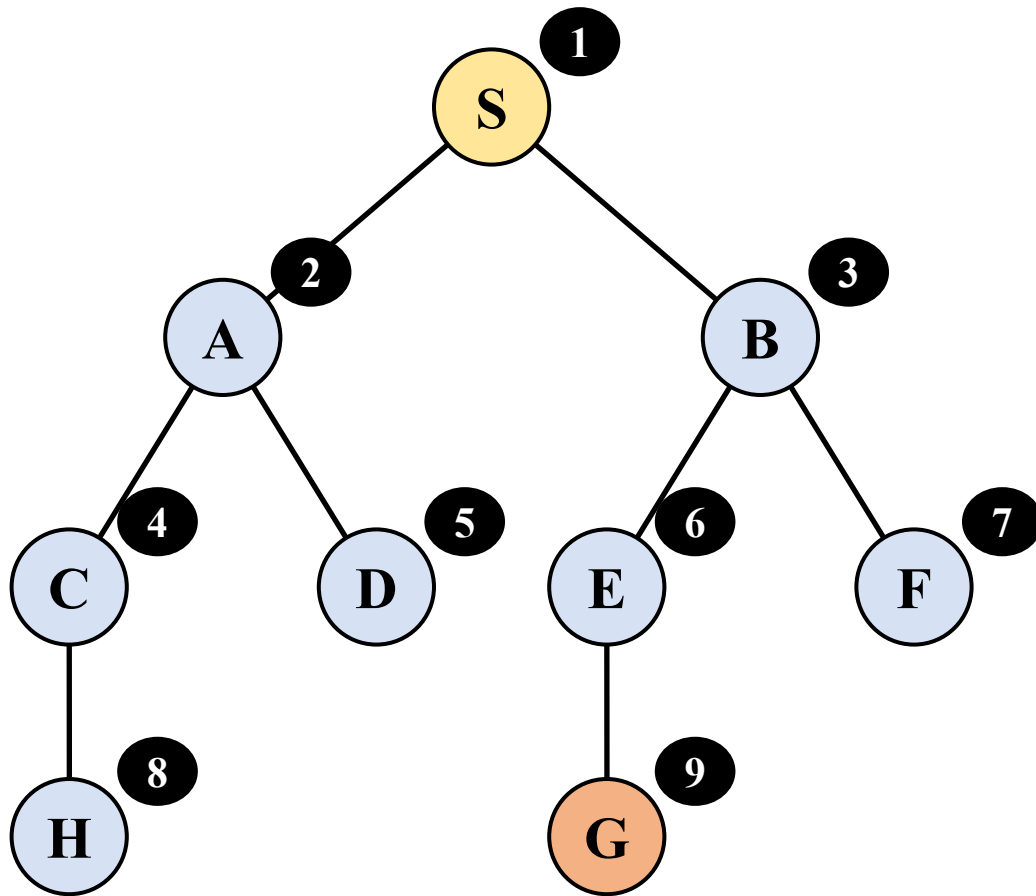
Informed (Heuristic) Search

- ▶ Uses problem-specific knowledge (a heuristic)
- ▶ Knows which state is more promising
- ▶ Examples: Greedy best-first, A* (later today)

Uninformed search algorithms covered today:

1. Breadth-First Search (BFS)
2. Depth-First Search (DFS)
3. Depth-Limited Search (DLS)
4. Iterative Deepening DFS (IDS)
5. Uniform-Cost Search (UCS)
6. Bidirectional Search

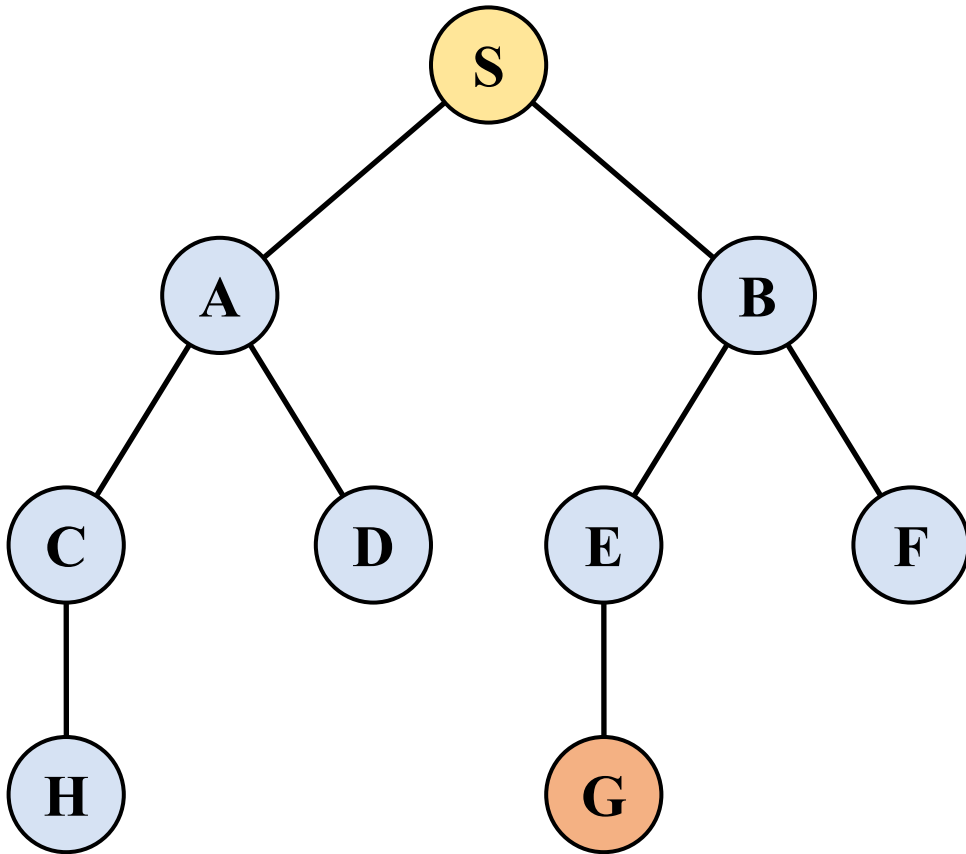
Breadth-First Search (BFS)



Black numbers = order of expansion

- ▶ Expand the shallowest unexpanded node first — level by level.
- ▶ Frontier is a FIFO queue: new children go to the back; the next node comes from the front.
- ▶ Expansion order:
 - S, A, B, C, D, E, F, H, G
- ▶ Solution: S → B → E → G

BFS: Step-by-Step Trace



Step	Expand	Frontier (queue: front → back)
1	S	A, B
2	A	B, C, D
3	B	C, D, E, F
4	C	D, E, F, H
5	D	E, F, H
6	E	F, H, G
7	F	H, G
8	H	G
9	G	Goal found!

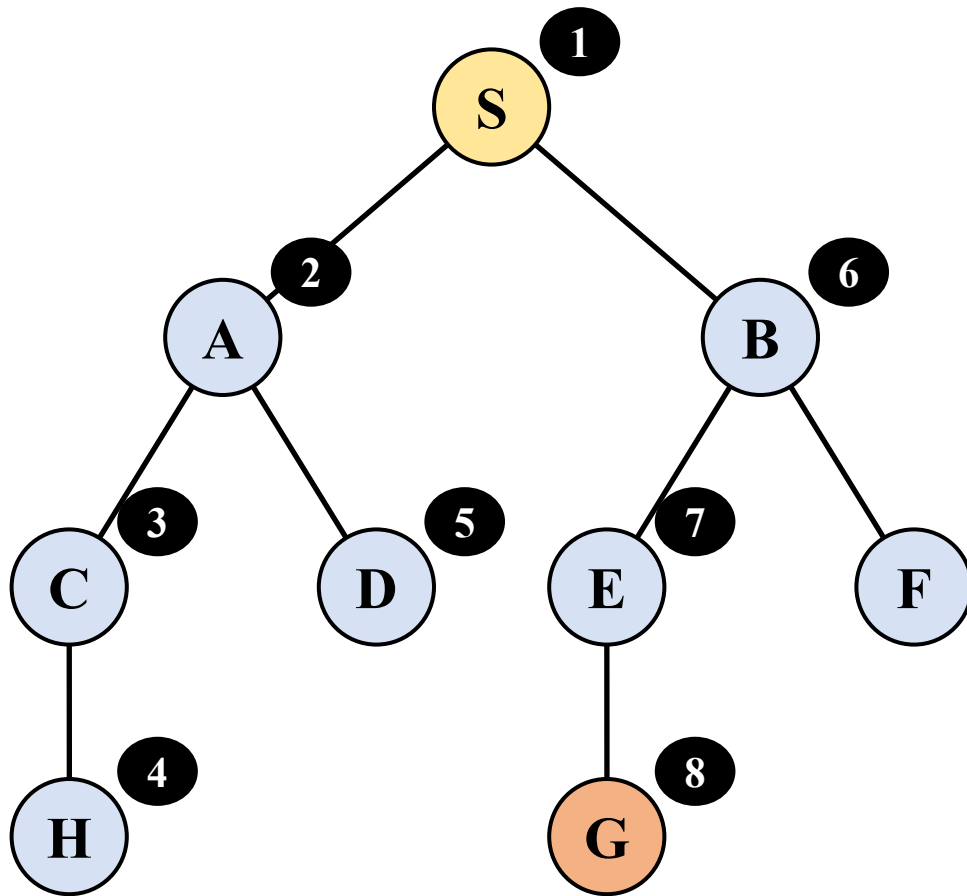
BFS: Properties

- ▶ **Complete:** Yes (if b is finite)
- ▶ **Optimal:** Yes, if all step costs are equal
- ▶ **Time:** $b + b^2 + \dots + b^d = O(b^d)$
- ▶ **Space:** $O(b^d)$ — keeps every node in memory
- ▶ **Advantage:** finds the shallowest solution (fewest steps)
- ▶ **Disadvantage:** needs huge memory; slow if the goal is deep

Depth	Nodes	Time	Memory
2	110	0.11 ms	107 KB
4	11,110	11 ms	10.6 MB
6	10^6	1.1 s	1 GB
8	10^8	2 min	103 GB
10	10^{10}	3 hours	10 TB
12	10^{12}	13 days	1 PB
14	10^{14}	3.5 years	99 PB

$b = 10$, 1 million nodes/s, 1 KB/node

Depth-First Search (DFS)



Black numbers = order of expansion

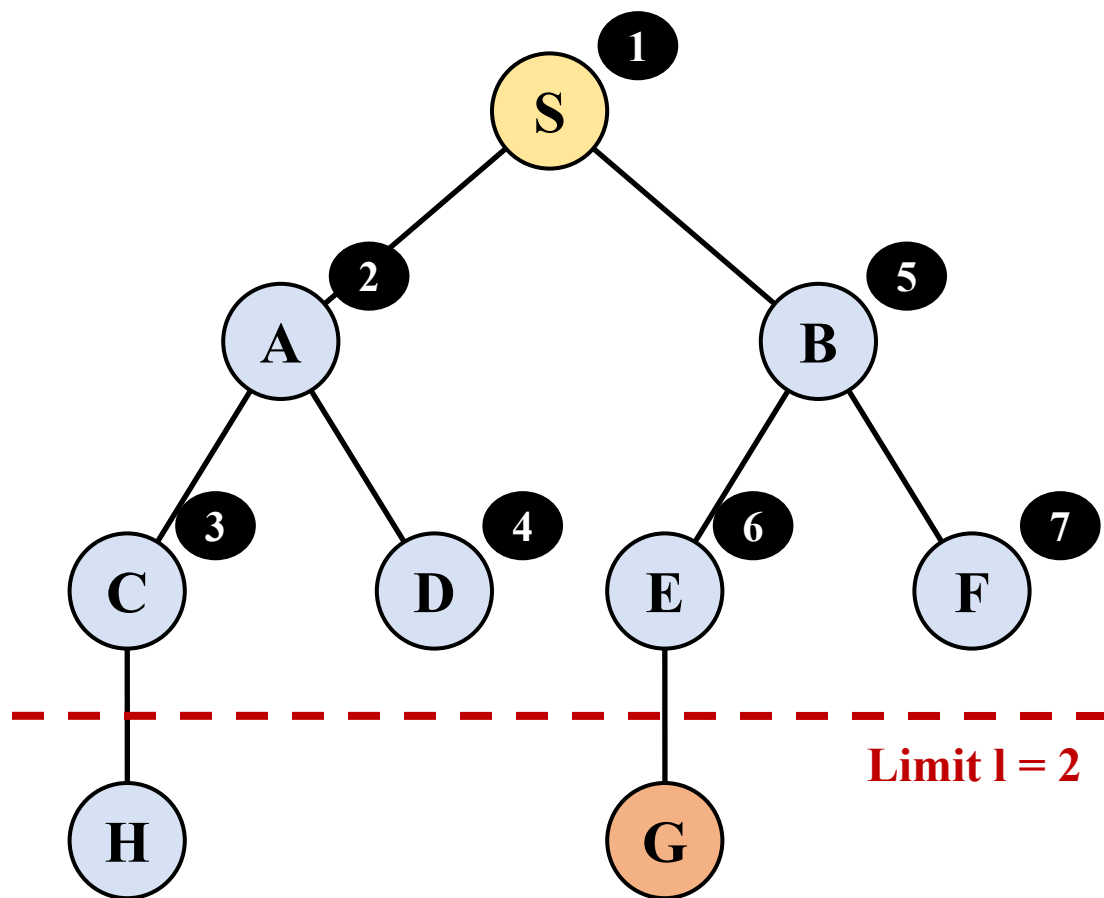
- ▶ Expand the deepest unexpanded node first.
- ▶ Frontier is a LIFO stack: the newest child is expanded next.
- ▶ Expansion order:
 - S, A, C, H, D, B, E, G
- ▶ Solution: S → B → E → G
- ▶ F is never expanded.

DFS: Properties & Backtracking

Property	DFS
Complete?	No — fails in infinite-depth spaces or with loops
Optimal?	No — may return a long solution
Time	$O(b^m)$ — terrible if m is much larger than d
Space	$O(bm)$ — only the current path and its siblings

- ▶ **Advantage:** very little memory; can be fast if it takes the right path.
- ▶ **Disadvantage:** can get lost in deep or infinite branches; no guarantee of the best solution.
- ▶ **Backtracking search:** a DFS variant that generates one successor at a time, so it needs only $O(m)$ memory.

Depth-Limited Search (DLS)



- ▶ DFS with a depth limit l : nodes at depth l are treated as having no children.
- ▶ Solves DFS's infinite-path problem.
- ▶ If $l < d \rightarrow$ incomplete (goal is too deep)
- ▶ If $l > d \rightarrow$ may not be optimal
- ▶ Time: $O(b^l)$ Space: $O(bl)$
- ▶ Example: with $l = 2$, G (depth 3) is not reached $\rightarrow$ returns cutoff

Iterative Deepening DFS (IDS)

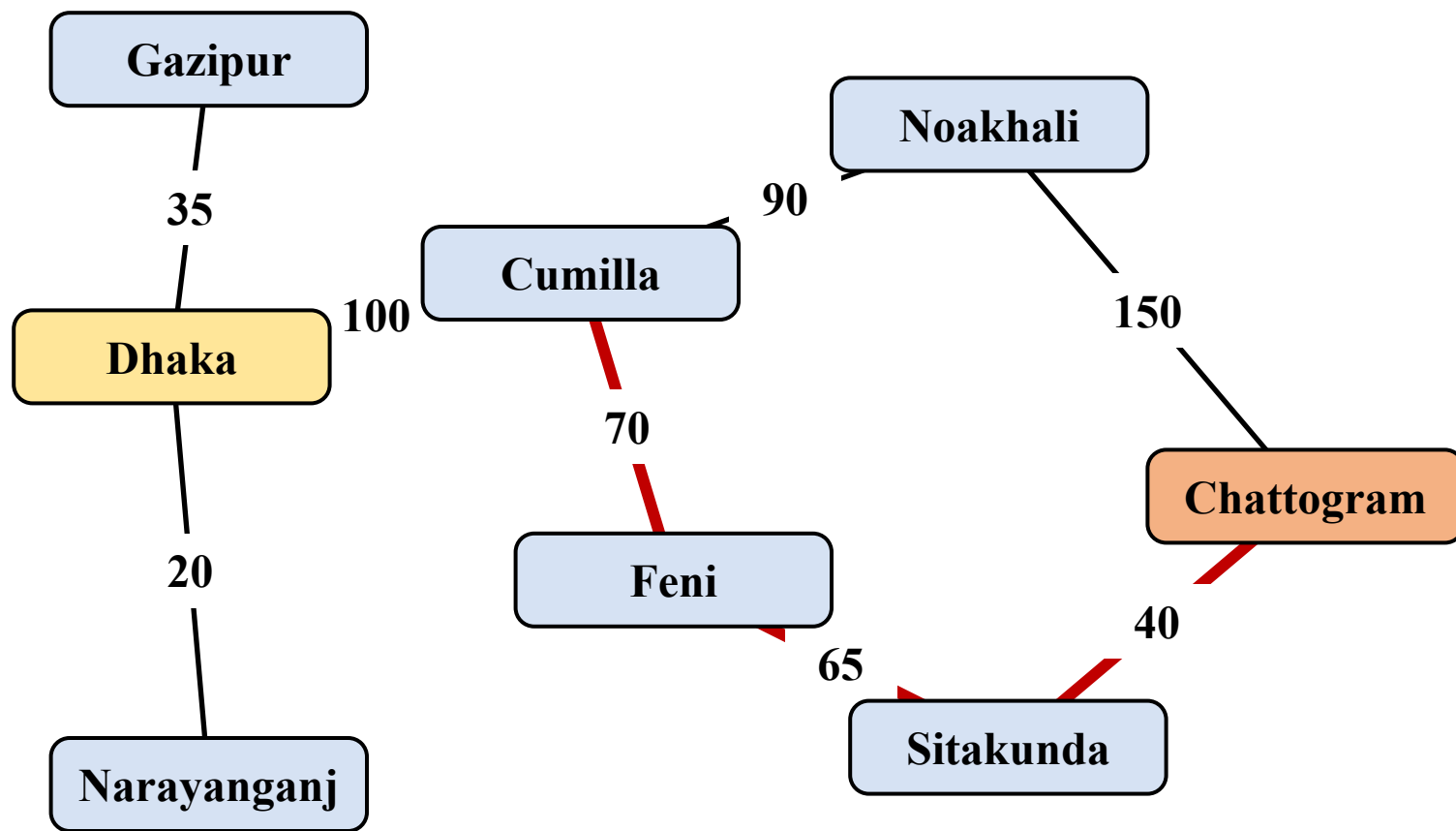
Limit	Nodes expanded (our example tree)
$l = 0$	S
$l = 1$	S, A, B
$l = 2$	S, A, C, D, B, E, F
$l = 3$	S, A, C, H, D, B, E, G → goal found

- ▶ Repeat depth-limited search with $l = 0, 1, 2, \dots$ until a goal is found.
- ▶ Combines BFS's completeness & optimality with DFS's low memory.
- ▶ Time: $O(b^d)$ Space: $O(bd)$
- ▶ $b = 10, d = 5$: $N(\text{IDS}) = 123,450$ vs $N(\text{BFS}) = 111,110$
- ▶ Preferred uninformed method when the search space is large and the depth of the solution is unknown.

Uniform-Cost Search (UCS)

- ▶ BFS finds the path with the fewest steps. When step costs differ, we want the cheapest path.
- ▶ UCS expands the node with the lowest path cost $g(n)$.
- ▶ Frontier is a priority queue ordered by $g(n)$.
- ▶ Goal test is applied when a node is selected for expansion, not when it is generated.
- ▶ If a cheaper path to a frontier node is found, it replaces the old one.
- ▶ Complete & optimal if every step cost $\geq \epsilon > 0$
- ▶ Time & space: $O(b^{1+\lfloor C^*/\epsilon \rfloor})$, where C^* = cost of the optimal solution
- ▶ Disadvantage: ignores the number of steps; can loop through zero-cost actions.

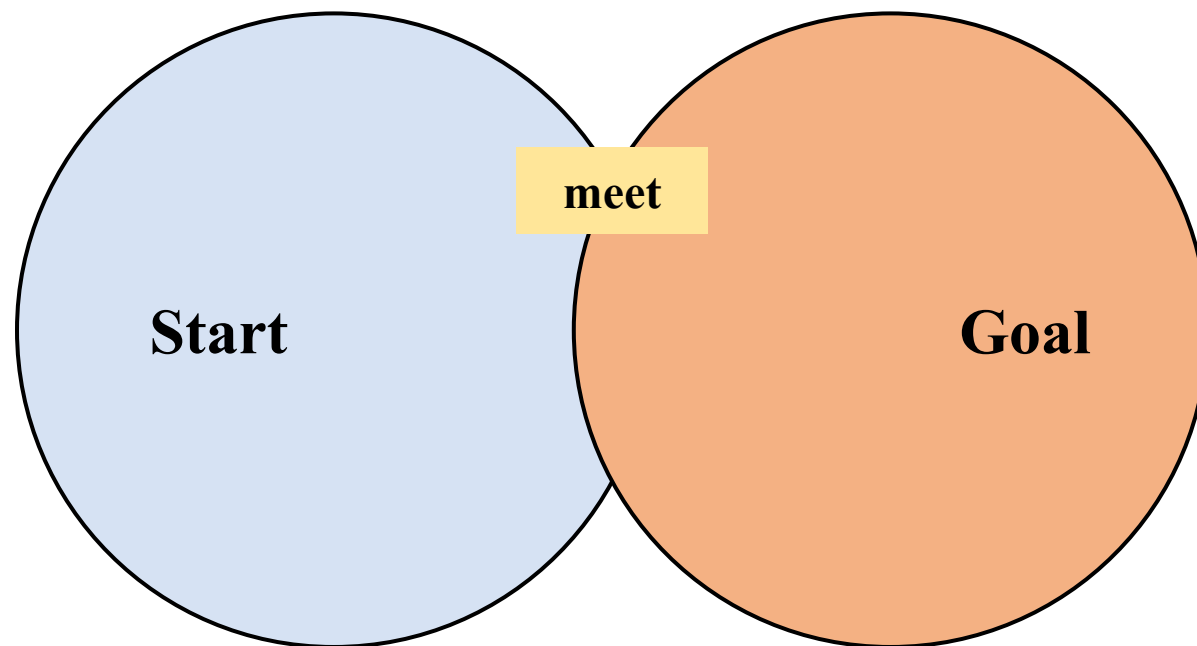
UCS Example: Dhaka → Chattogram



#	Expand (g)	Added / updated
1	Dhaka 0	Narayanganj 20, Gazipur 35, Cumilla 100
2	Narayanganj 20	—
3	Gazipur 35	—
4	Cumilla 100	Feni 170, Noakhali 190
5	Feni 170	Sitakunda 235
6	Noakhali 190	Chattogram 340
7	Sitakunda 235	Chattogram 275 (cheaper!)
8	Chattogram 275	Goal ✓

Optimal path: Dhaka → Cumilla → Feni → Sitakunda → Chattogram = 275 km

Bidirectional Search



- ▶ Run two searches at once: forward from the start and backward from the goal.
- ▶ Stop when the two frontiers meet.
- ▶ $b^{d/2} + b^{d/2}$ is much smaller than b^d
- ▶ $b = 10, d = 6$: 2,220 nodes vs 1,111,110 for BFS
- ▶ Time & space: $O(b^{d/2})$
- ▶ Needs a known goal state and the ability to search backwards.

Comparing Uninformed Search Strategies

Criterion	BFS	UCS	DFS	DLS	IDS	Bidirectional
Complete?	Yes (a)	Yes (a, b)	No	No	Yes (a)	Yes (a, d)
Time	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(b^m)$	$O(b^l)$	$O(b^d)$	$O(b^{d/2})$
Space	$O(b^d)$	$O(b^{1+\lfloor C^*/\epsilon \rfloor})$	$O(bm)$	$O(bl)$	$O(bd)$	$O(b^{d/2})$
Optimal?	Yes (c)	Yes	No	No	Yes (c)	Yes (c, d)

(a) complete if b is finite (b) if every step cost $\geq \epsilon > 0$

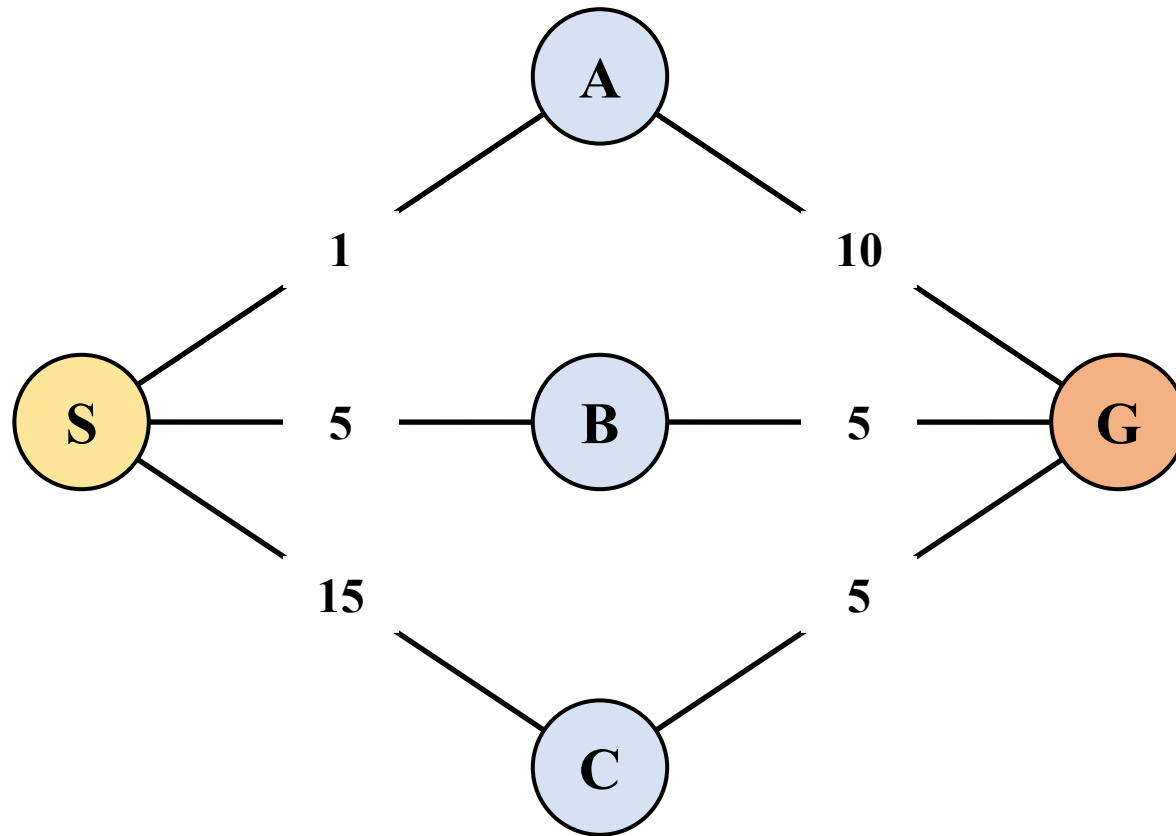
(c) optimal if all step costs are identical (d) if both directions use BFS

b = branching factor, d = depth of shallowest goal, m = maximum depth, l = depth limit, C^* = optimal cost

Which Strategy Should We Use?

Situation	Good choice	Why
All step costs equal, enough memory	BFS	Finds the shallowest solution
Step costs differ (distances, prices)	UCS	Finds the cheapest solution
Very little memory, many solutions	DFS	Needs only $O(bm)$ memory
Maximum useful depth is known	DLS	Avoids infinite branches
Large space, solution depth unknown	IDS	Complete, optimal, low memory
Single known goal, reversible actions	Bidirectional	Searches only $O(b^{d/2})$ nodes

Class Exercise (5 minutes)



- ▶ Start = S, Goal = G. Numbers are step costs.
- ▶ Expand children in alphabetical order.
- ▶ Find the path and its cost using:
 - Breadth-First Search
 - Depth-First Search
 - Uniform-Cost Search
- ▶ Which one gives the cheapest path? Why?

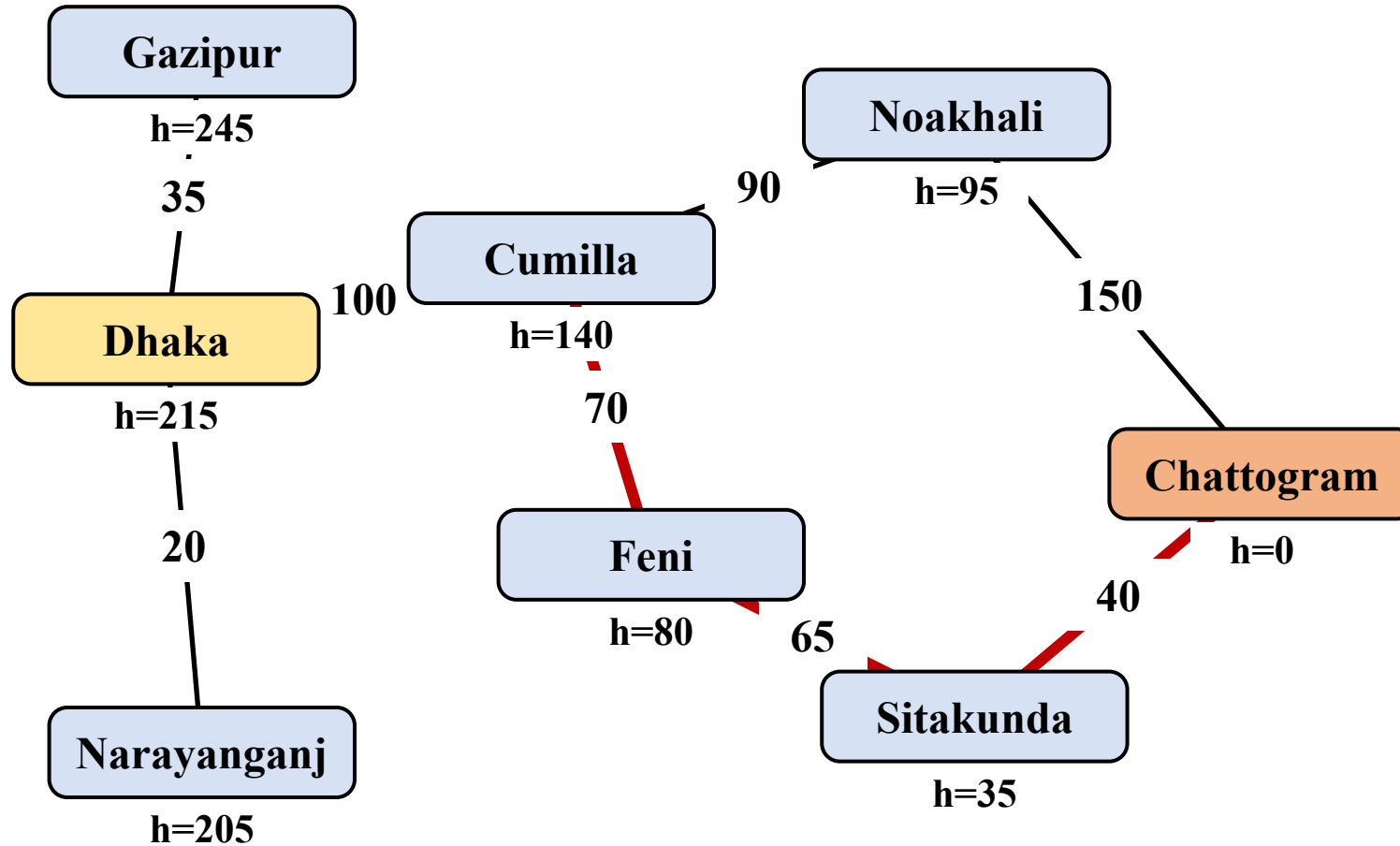
Informed Search: The Heuristic Function

- ▶ A heuristic function $h(n)$ estimates the cost of the cheapest path from node n to a goal.
- ▶ $h(\text{goal}) = 0$
- ▶ It uses problem-specific knowledge (e.g. straight-line distance on a map).
- ▶ Search algorithms choose the node with the lowest evaluation function $f(n)$:
 - Uniform-Cost: $f(n) = g(n)$
 - Greedy Best-First: $f(n) = h(n)$
 - A*: $f(n) = g(n) + h(n)$

City	$h(n)$ to Chattogram
Dhaka	215
Gazipur	245
Narayanganj	205
Cumilla	140
Noakhali	95
Feni	80
Sitakunda	35
Chattogram	0

h = approximate straight-line distance (km)

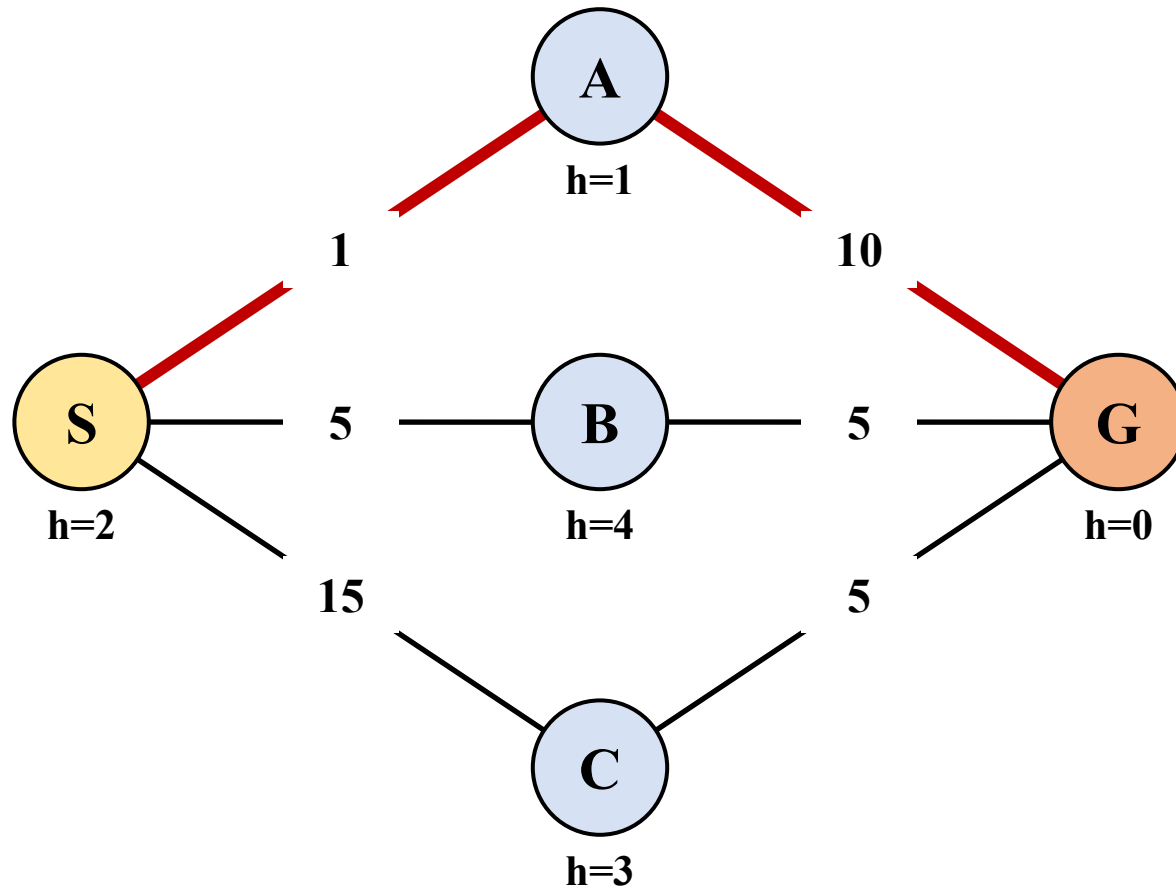
Greedy Best-First Search



#	Expand (h)	Added to frontier (h)
1	Dhaka 215	Narayanganj 205, Cumilla 140, Gazipur 245
2	Cumilla 140	Feni 80, Noakhali 95
3	Feni 80	Sitakunda 35
4	Sitakunda 35	Chattogram 0
5	Chattogram 0	Goal ✓

Expand the node with the lowest $h(n)$. Here: 275 km with only 5 expansions (UCS: 8).

Greedy Best-First: Fast but Not Optimal



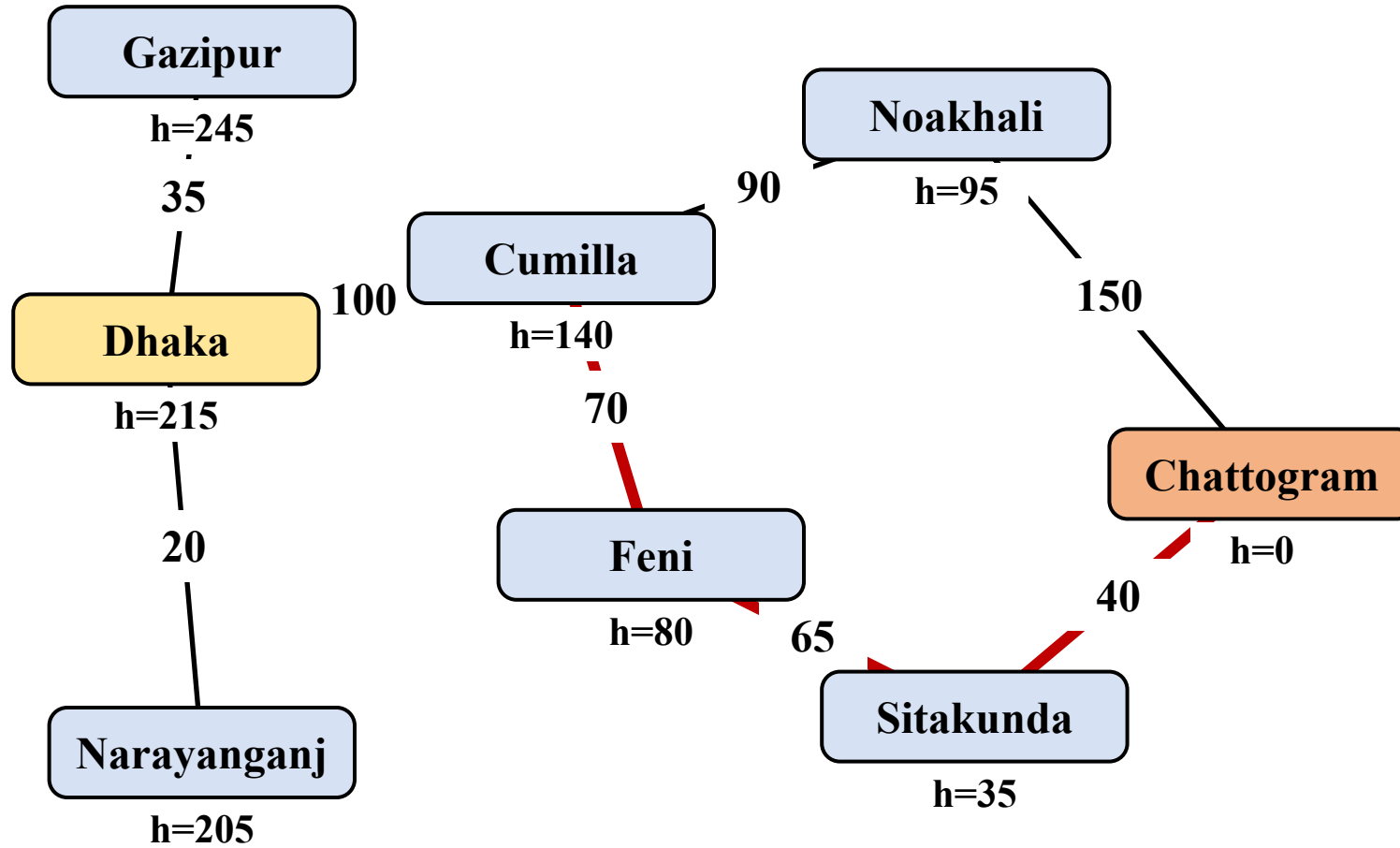
- ▶ Greedy expands $S \rightarrow A$ ($h = 1$, looks closest) $\rightarrow G$
- ▶ Returns $S \rightarrow A \rightarrow G$, cost 11
- ▶ The best path is $S \rightarrow B \rightarrow G$, cost 10 ✗
- ▶ Optimal? No — it ignores the cost already paid, $g(n)$
- ▶ Complete? No in tree search (can loop); yes in finite graph search
- ▶ Time & space: $O(b^m)$ worst case; a good heuristic reduces this a lot

A* Search

$$f(n) = g(n) + h(n)$$

- ▶ **g(n)** = actual cost from the start to n
- ▶ **h(n)** = estimated cost from n to the goal
- ▶ **f(n)** = estimated total cost of the best solution through n
- ▶ Expand the node with the **lowest f(n)** (priority queue) — like UCS, but guided by h.
- ▶ **Complete:** Yes (finite b, step costs $\geq \epsilon > 0$)
- ▶ **Optimal:** Yes, if h is **admissible** (tree search) or **consistent** (graph search)
- ▶ Used in navigation apps, games and robot path planning.

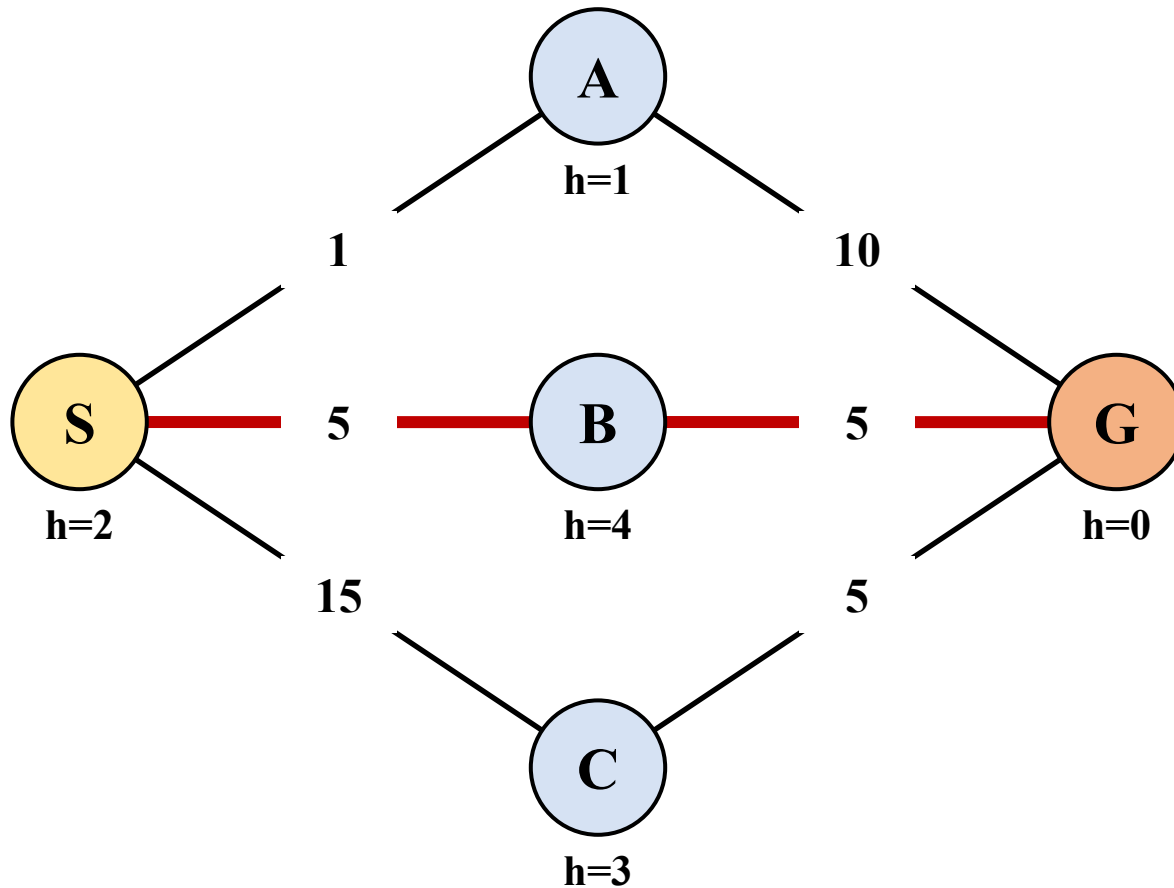
A* Example: Dhaka → Chattogram



#	Expand ($f = g + h$)	Added (f)
1	Dhaka $0+215 = 215$	Narayanganj 225, Cumilla 240, Gazipur 280
2	Narayanganj 225	—
3	Cumilla $100+140 = 240$	Feni 250, Noakhali 285
4	Feni $170+80 = 250$	Sitakunda 270
5	Sitakunda $235+35 = 270$	Chattogram 275
6	Chattogram $275+0 = 275$	Goal ✓

**Optimal: 275 km with 6 expansions
(UCS: 8)**

A* Avoids the Greedy Trap



#	Expand (f)	Added / updated ($f = g + h$)
1	S $0+2 = 2$	A $1+1 = 2$, B $5+4 = 9$, C $15+3 = 18$
2	A 2	G $11+0 = 11$
3	B 9	G $10+0 = 10$ (cheaper!)
4	G 10	Goal ✓

- ▶ A* returns $S \rightarrow B \rightarrow G$, cost 10 — optimal ✓
- ▶ Greedy returned $S \rightarrow A \rightarrow G$, cost 11

Admissible and Consistent Heuristics

- ▶ **Admissible**: never overestimates the true cost: $h(n) \leq h^*(n)$
 - Feni: $h = 80 \leq \text{true cost } 105$ ✓
- ▶ **Consistent**: $h(n) \leq c(n, n') + h(n')$ (triangle inequality)
 - Cumilla: $140 \leq 70 + h(\text{Feni}) = 150$ ✓
- ▶ Every consistent heuristic is also admissible.
- ▶ A* is optimal with an **admissible** h (tree search) or a **consistent** h (graph search).
- ▶ If h overestimates, A* may return a **non-optimal** path.

City	$h(n)$	$h^*(n)$
Dhaka	215	275
Gazipur	245	310
Narayanganj	205	295
Cumilla	140	175
Noakhali	95	150
Feni	80	105
Sitakunda	35	40
Chattogram	0	0

$h^*(n)$ = true shortest road distance

Heuristics for the 8-Puzzle

2	8	3
1	6	4
7		5

Start

1	2	3
8		4
7	6	5

Goal

- ▶ **h1** = number of misplaced tiles
- ▶ **h2** = Manhattan distance: sum of each tile's horizontal + vertical distance to its goal square
- ▶ Both are **admissible**; $h2 \geq h1$, so **h2 dominates h1** and A* with **h2** expands fewer nodes.

Tile	1	2	3	4	5	6	7	8	Total
h1: misplaced?	1	1	0	0	0	1	0	1	4
h2: distance	1	1	0	0	0	1	0	2	5

True optimal solution = 5 moves, so h2 is exact here and h1 underestimates.

Comparing UCS, Greedy Best-First and A*

Algorithm	Expands lowest	Complete?	Optimal?	Dhaka → Chattogram	Exercise graph
Uniform-Cost	$g(n)$	Yes	Yes	8 expansions, 275 km	S→B→G, cost 10 ✓
Greedy Best-First	$h(n)$	No (tree)	No	5 expansions, 275 km	S→A→G, cost 11 ✗
A*	$g(n) + h(n)$	Yes	Yes (admissible h)	6 expansions, 275 km	S→B→G, cost 10 ✓

- ▶ A* keeps the optimality of UCS but uses h to expand fewer nodes.
- ▶ The better (closer to true cost) the admissible heuristic, the faster A* runs.
- ▶ Next lecture: more heuristic search — Hill Climbing, AND–OR graphs and AO*.

Summary

- ▶ Search algorithms differ only in which frontier node they expand next.
- ▶ BFS (FIFO queue) and UCS (priority queue by g) are complete and optimal under their conditions but use a lot of memory.
- ▶ DFS and DLS use little memory but are not complete or optimal; IDS is usually the best uninformed method.
- ▶ Informed search uses a heuristic $h(n)$: Greedy expands lowest h (fast, not optimal); A^* expands lowest $g + h$.
- ▶ A^* is optimal when h is admissible / consistent; better heuristics (e.g. Manhattan distance) mean fewer expansions.
- ▶ Reading: Russell & Norvig, AIMA, Chapter 3 (3.3–3.6).

Thank You

Any questions?